

SECTION 36 — MOCK INTERVIEW SETS

Purpose: Structural evaluation suites and mock interview loops matching Deloitte, Big 4, Tier-1 Tech, and FAANG Senior SDET assessment criteria. Designed to test technical depth, coding rigor, architectural problem solving, and behavioral alignment.

36.1 Java + Coding Mock

Question 1: HashMap Concurrent Modification & Custom Key Contract

Time Allocation: 6 mins

Interviewer Prompt: "Suppose we create a `HashMap<CustomKey, String>`. `CustomKey` has `id` (int) and `name` (String). We override `equals()` and `hashCode()`. After inserting an entry into the map, we mutate `key.setName("NewName")` and then call `map.get(key)`. What happens and why? How would you fix this, and how does `HashMap` handle hash collisions internally in Java 8+?"

Expected Answer (Senior Level):

- Lookup Failure & Silent Leak:** `map.get(key)` returns `null`. `HashMap` determines bucket index using `key.hashCode()`. When `name` was mutated, the hash code changed. The lookup recalculates `hashCode()` based on the new `name`, mapping to a completely different bucket. The original entry remains orphaned in the old bucket, causing a silent memory leak.
- Fix:** Keys in hash-based collections MUST be immutable. Mark fields `private final`, omit setters, and initialize via constructor.
- Internal Mechanics (Java 8+):**
 - Hash computation: `h = key.hashCode() ^ (h >>> 16)` (spread high bits to low).
 - Bucket index: `(n - 1) & hash`.
 - Collision resolution: Linked list per bucket. When bucket size reaches `TREEIFY_THRESHOLD = 8` AND total capacity ≥ 64 , the bucket transforms from a singly-linked list into a Red-Black Tree ($O(N)$ to $O(\log N)$). When shrinking below `UNTREEIFY_THRESHOLD = 6`, it converts back to a linked list.

```
// Immutable Key Implementation Fix
public final class CustomKey {
    private final int id;
    private final String name;

    public CustomKey(int id, String name) {
        this.id = id;
        this.name = Objects.requireNonNull(name, "Name cannot be null");
    }

    public int getId() { return id; }
    public String getName() { return name; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        CustomKey customKey = (CustomKey) o;
```

```

        return id == customKey.id && Objects.equals(name, customKey.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id, name);
    }
}

```

Evaluation Criteria:

- Identifies returning `null` and silent memory retention.
- Mentions object immutability (`final` fields, no setters).
- Explains bucket index calculation (`n - 1`) & hash .
- Articulates treeification thresholds (`TREEIFY_THRESHOLD = 8` , `capacity >= 64` , Red-Black tree conversion).

Red Flags:

- Thinks mutating a key throws `ConcurrentModificationException` .
- Unaware of Java 8 Red-Black tree bin conversion ($O(N)$ \to $O(\log N)$).

Question 2: Memory Leak Diagnosis in Automation Frameworks

Time Allocation: 8 mins

Interviewer Prompt: "During a 2,000 E2E parallel test execution suite, JVM memory consumption grows linearly until an `OutOfMemoryError: Java heap space` is thrown. Thread dumps show 500+ `ThreadLocal` instances holding browser driver drivers or extent reports. What is the root cause and how do you remediate it?"

Expected Answer (Senior Level):

1. **Root Cause:** Thread pooling (e.g., TestNG or JUnit 5 parallel thread pools). When threads are reused across tests, value references inside `ThreadLocal<WebDriver>` or `ThreadLocal<ExtentTest>` persist in the thread's `ThreadLocalMap` . Even if the test finishes, the thread remains alive in the pool, holding references to heavy objects (driver sessions, page objects, DOM snapshots), preventing Garbage Collection.
2. **Remediation:**
 - Always invoke `threadLocal.remove()` in an `@AfterMethod` / `@AfterEach` or `try-finally` block. Calling `.set(null)` is insufficient because the `ThreadLocal` entry key remains in the `ThreadLocalMap` . `.remove()` explicitly purges the entry key and value from the array.

```

public class DriverManager {
    private static final ThreadLocal<WebDriver> driverTL = new ThreadLocal<>();

    public static WebDriver getDriver() {
        return driverTL.get();
    }

    public static void setDriver(WebDriver driver) {

```

```

        driverTL.set(driver);
    }

    public static void unload() {
        if (driverTL.get() != null) {
            driverTL.get().quit();
            driverTL.remove(); // CRITICAL: Purges ThreadLocal entry preventing pool
retention leak
        }
    }
}

```

Evaluation Criteria:

- Distinguishes between thread termination vs thread pool thread reuse.
- Explains `ThreadLocalMap` lifecycle tied to thread life span.
- Emphasizes explicit `.remove()` call instead of setting null.

Red Flags:

- Recommends increasing heap size (`-Xmx`) as the solution.
- Confuses `ThreadLocal` with static synchronization.

36.2 Selenium + TestNG Mock

Question 1: StaleElementReferenceException Architectural Solution

Time Allocation: 6 mins

Interviewer Prompt: "Why does `StaleElementReferenceException` occur in Selenium, and how do you build a framework-level resilience layer to handle it without wrapping every test in `try-catch`?"

Expected Answer (Senior Level):

1. **Root Cause:** The element reference ID assigned by WebDriver (W3C element UUID) is no longer valid in the browser's current DOM tree. This happens when the DOM nodes are re-rendered (AJAX reload, Angular/React state update, DOM replacement) even if the element looks identical on screen.
2. **Framework Resilience Strategy:**
 - **Page Object Dynamic Locators (Re-querying):** Avoid storing `WebElement` fields. Store `By` locators or use Page Factory proxy / custom wrapper methods that re-evaluate `driver.findElement(by)` dynamically at click/type execution time.
 - **Fluent Explicit Retry Wrapper:** Implement a generic explicit wait or retry wrapper catching `StaleElementReferenceException` up to a configured timeout limit.

```

public class FluentActions {
    private final WebDriver driver;
    private final Duration timeout = Duration.ofSeconds(10);
    private final Duration polling = Duration.ofMillis(500);

    public FluentActions(WebDriver driver) {
        this.driver = driver;
    }
}

```

```

    }

    public void clickWithRetry(By locator) {
        new FluentWait<>(driver)
            .withTimeout(timeout)
            .pollingEvery(polling)
            .ignoring(StaleElementReferenceException.class)
            .ignoring(ElementClickInterceptedException.class)
            .until(d -> {
                WebElement element = d.findElement(locator);
                element.click();
                return true;
            });
    }
}

```

Evaluation Criteria:

- Explains element DOM detachment / node replacement mechanics.
- Advocates storing `By` locators over persistent `WebElement` instances.
- Constructs clean `FluentWait` with `.ignoring(StaleElementReferenceException.class)` .

Red Flags:

- Suggests adding `Thread.sleep()` .
- Catches `Exception` globally without specific retry logic.

36.3 REST Assured Mock

Question 1: Dynamic POJO Serialization & Response Contract Validation

Time Allocation: 7 mins

Interviewer Prompt: "How do you handle dynamic JSON payloads where fields vary based on tenant configuration, and how do you enforce strict API schema contracts alongside functional assertions in REST Assured?"

Expected Answer (Senior Level):

1. Dynamic Payload Serialization:

- Use **Jackson/Gson annotations** such as `@JsonInclude(JsonInclude.Include.NON_NULL)` and `@JsonAnyGetter` / `@JsonAnySetter` to handle variable key-value pairs without creating rigid POJO hierarchies.
- Use **Builder Pattern** for payload creation with dynamic method chaining.

2. Schema & Contract Enforcement:

- Combine functional `JSONPath` assertions with structural JSON Schema Validation using `io.restassured.module.json.schema.JsonSchemaValidator.matchesJsonSchemaInClasspath()` .

```

@JsonInclude(JsonInclude.Include.NON_NULL)
public class CreateUserPayload {
    private String name;
}

```

```

private String role;
private Map<String, Object> dynamicAttributes = new HashMap<>();

// Builder Pattern
public static class Builder {
    private String name;
    private String role;
    private Map<String, Object> dynamicAttributes = new HashMap<>();

    public Builder withName(String name) { this.name = name; return this; }
    public Builder withRole(String role) { this.role = role; return this; }
    public Builder addAttribute(String key, Object val) {
        this.dynamicAttributes.put(key, val);
        return this;
    }
}

public CreateUserPayload build() {
    CreateUserPayload payload = new CreateUserPayload();
    payload.name = this.name;
    payload.role = this.role;
    payload.dynamicAttributes = this.dynamicAttributes;
    return payload;
}

}

@JsonAnyGetter
public Map<String, Object> getDynamicAttributes() { return dynamicAttributes; }
}

// REST Assured Verification
given()
    .contentType(ContentType.JSON)
    .body(payload)
.when()
    .post("/api/v1/users")
.then()
    .statusCode(201)
    .body("id", notNullValue())
    .body(matchesJsonSchemaInClasspath("schemas/user-created-schema.json"));

```

Evaluation Criteria:

- Demonstrates Builder pattern and Jackson dynamic annotations.
- Explains decoupling payload construction from test body.
- Integrates JSON Schema validation alongside status/field assertions.

Red Flags:

- Manually concatenates JSON strings using String format or `+`.
- Performs no schema validation, only checking 1-2 status codes.

36.4 Playwright + TypeScript Mock

Question 1: Custom Fixtures & Authentication State Injection

Time Allocation: 8 mins

Interviewer Prompt: "In Playwright with TypeScript, how do you eliminate login overhead across 500 test cases using `storageState` and custom fixture extension?"

Expected Answer (Senior Level):

1. **Global Auth Setup (`storageState`):** Perform authentication once in a global setup project or setup test file, saving authenticated storage state (cookies, localStorage) to disk (`playwright/.auth/user.json`).
2. **Fixture Dependency Injection:** Extend base `test` with custom page object fixtures so tests auto-receive fully authenticated, pre-initialized Page Object instances without boilerplate setup.

```
// auth.setup.ts
import { test as setup, expect } from '@playwright/test';

const authFile = 'playwright/.auth/user.json';

setup('authenticate', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Username').fill(process.env.TEST_USER!);
  await page.getByLabel('Password').fill(process.env.TEST_PASS!);
  await page.getByRole('button', { name: 'Sign in' }).click();
  await page.waitForURL('/dashboard');
  await page.context().storageState({ path: authFile });
});

// fixtures/test-fixtures.ts
import { test as base } from '@playwright/test';
import { DashboardPage } from '../pages/DashboardPage';

type MyFixtures = {
  dashboardPage: DashboardPage;
};

export const test = base.extend<MyFixtures>({
  dashboardPage: async ({ page }, use) => {
    const dashboard = new DashboardPage(page);
    await dashboard.goto();
    await use(dashboard);
  },
});

export { expect } from '@playwright/test';
```

Evaluation Criteria:

- Demonstrates `storageState` flow to eliminate repeated UI login steps.
- Uses Playwright `test.extend<T>()` for clean Page Object Dependency Injection.

- Uses web-first locators (`getByRole` , `getByLabel`).

Red Flags:

- Executes UI login inside `beforeEach` for every single test.
- Uses hardcoded `page.waitForTimeout()` instead of auto-waiting locators.

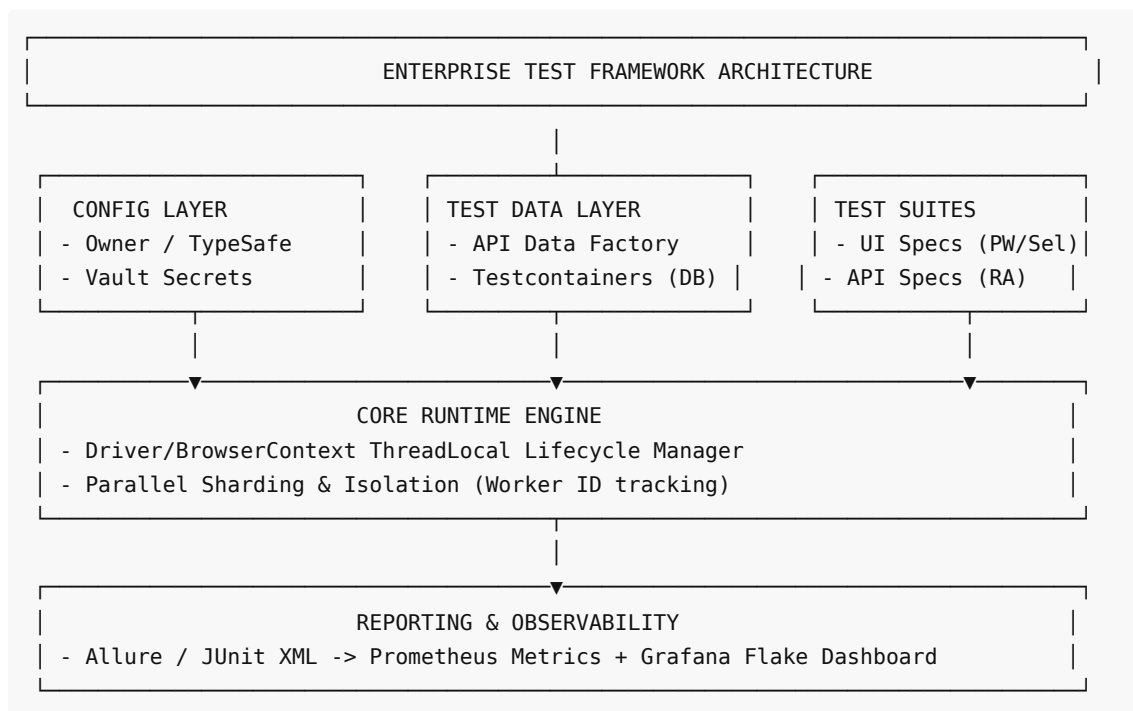
36.5 Framework Design Mock

Question 1: Enterprise Multi-Tenant Test Framework Architecture

Time Allocation: 10 mins

Interviewer Prompt: "Design a test framework supporting 5,000+ tests executing across 4 environments (DEV, QA, STG, PROD) with multi-region tenant requirements. Detail configuration management, data isolation, and reporting strategy."

Expected Answer (Senior Level):



- 1. Config Management:** Use TypeSafe Config / Owner library reading system properties (`- Denv=STG -Dtenant=US_EAST`). Store credentials in HashiCorp Vault / AWS Secrets Manager injected at runtime.
- 2. Test Data Isolation:** Avoid shared test accounts. Use an **API Data Factory** to dynamically provision ephemeral users/entities per worker (`Worker ID + Timestamp + UUID`). Clean up via API teardown hooks or isolated Testcontainers.
- 3. Reporting & Flake Analytics:** Generate standard JUnit XML reports parsed into a centralized Grafana dashboard to track flakiness by test ID, worker ID, and environment over time.

36.6 CI/CD Mock

Question 1: Sharded GitHub Actions Pipeline Optimization

Time Allocation: 8 mins

Interviewer Prompt: "Your E2E test suite takes 45 minutes to execute on CI. How do you design a GitHub Actions workflow to run this in under 6 minutes with proper artifact handling?"

Expected Answer (Senior Level):

1. **Parallel Matrix Sharding:** Use GitHub Actions matrix strategy to split test execution across `$N$` parallel runner nodes (e.g., 8 shards). Pass `--shard=${{ matrix.shard }}/${{ strategy.total-shards }}` to Playwright or custom TestNG listener partitioning.
2. **Fail-Fast Early Stages:** Run linting, security scanning (Trivy), and fast unit/API tests before launching heavy E2E matrix shards.
3. **Report Merging:** Merge shard trace/XML artifacts into a single unified report using `actions/upload-artifact` and `@playwright/test` blob reporter merge.

```
name: Parallel E2E Suite
on:
  push:
    branches: [ main ]
  pull_request:

jobs:
  e2e-tests:
    timeout-minutes: 15
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        shard: [1, 2, 3, 4, 5, 6, 7, 8]
        total-shards: [8]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 18
          cache: 'npm'
      - run: npm ci
      - run: npx playwright install --with-deps chromium
      - run: npx playwright test --shard=${{ matrix.shard }}/${{ matrix.total-shards }}
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: blob-report-${{ matrix.shard }}
          path: blob-report/
          retention-days: 7
```

36.7 Debugging Mock

Question 1: Intermittent CI Failure Triage (Race Conditions)

Time Allocation: 7 mins

Interviewer Prompt: "A critical checkout test fails 5% of the time on CI with `TimeoutError: element not visible`, but passes 100% locally. Walk me through your step-by-step diagnostic workflow."

Expected Answer (Senior Level):

1. **Trace Viewer & Network Logs:** Download the failure trace (`trace.zip`) or video artifact from CI. Inspect DOM snapshot, console logs, and network waterfall at the exact point of failure.
 2. **Identify Async Race Condition:** Check if a background API response (e.g., payment gateway check or inventory validation) finishes AFTER the element visibility check.
 3. **Remediation:** Replace static/generic waits with explicit network response waits (`page.waitForResponse('/api/checkout/validate')`) or dynamic web-first assertions (`expect(locator).toBeVisible()`) that wait for state stability.
-

36.8 Senior SDET Scenario Mock

Question 1: Flaky Test Reduction Plan (From 12% to < 1%)

Time Allocation: 8 mins

Interviewer Prompt: "You join a team where 12% of CI builds fail due to flaky tests. Developers ignore test failures and re-run pipelines until green. How do you resolve this culturally and technically within 30 days?"

Expected Answer (Senior Level):

1. **Immediate Quarantine (Day 1-5):** Implement a quarantine process (`@Flaky` tag or dedicated pipeline bucket). Move flaky tests out of blocking PR gates immediately to restore pipeline trust.
 2. **Root Cause Categorization (Day 6-15):** Analyze failure signatures into 4 categories:
 - *Shared Data Collisions:* Tests mutating shared DB state simultaneously.
 - *Async Synchronization:* Missing auto-waits or network response hooks.
 - *Environment Instability:* Unstable external dependencies (stub with WireMock/MSW).
 - *Order Dependency:* Tests relying on execution sequence (randomize order).
 3. **Technical Remediation (Day 16-25):** Enforce strict PR checks: mandatory 50x parallel retry dry-runs for new tests before merging. Max 1 automatic retry on CI.
 4. **Governance SLA (Day 26-30):** Track Flake SLA metric (< 1%). Untriaged flaky tests automatically get converted into Jira bugs assigned to feature teams.
-

36.9 Project Deep-Dive Mock

Question 1: Architecture Defense & Modernization Trade-offs

Time Allocation: 10 mins

Interviewer Prompt: "Tell me about your most complex automation project. What major architectural trade-offs did you make, what failed, and what would you design differently today?"

Expected Answer (Senior Level):

- **Structure response using STAR+ (Situation, Task, Action, Result + Lessons Learned).**
- **Example Response Pattern:**
 - *Context:* Modernized legacy Selenium/Java framework executing 4,000 tests taking 3.5 hours down to Playwright/TypeScript sharded suite taking 8 minutes.
 - *Trade-off:* Moving from Java to TypeScript required upskilling QA team, but gave 10x faster execution and native browser context isolation.
 - *What Failed:* Initially attempted to migrate all 4,000 tests 1:1, including obsolete UI tests. Refactored strategy to replace 60% of UI tests with fast API contract specs.
 - *Result:* Reduced execution time by 92%, cut pipeline cloud runner costs by 65%, reduced flake rate from 14% to 0.4%.

36.10 Full Deloitte Mock Interview

Interview Loop Architecture (60 Minutes)

Segment	Duration	Focus Area
1. Introduction & Background	5 mins	Career trajectory, senior SDET scope, enterprise scale
2. Technical Core & Coding	20 mins	Java/TS mechanics, live coding challenge, data structures
3. Framework & Architecture	15 mins	System design, CI/CD sharding, multi-environment design
4. Senior Leadership & Scenarios	10 mins	Flaky test triage, shift-left governance, developer coaching
5. Candidate Q&A	10 mins	Technical culture, Deloitte client project engagement model

Live Coding Challenge (Segment 2 - 15 mins)

Problem Statement: Given an array of HTTP response logs representing API response times in milliseconds [120, 450, 800, 230, 990, 150, 450, 300] , write a clean TypeScript/Java function to calculate:

1. The **p95 (95th percentile)** response time.
2. Filter out outliers ($\$ > 800\text{ms}$) and return unique response times sorted in descending order.

```
function analyzeResponseTimes(logs: number[]): { p95: number; filteredUnique: number[] } {  
  if (!logs || logs.length === 0) {  
    throw new Error("Log array cannot be empty");  
  }  
  
  // 1. Sort ascending for percentile calculation  
  const sorted = [...logs].sort((a, b) => a - b);
```

```
// 2. Calculate p95 index (Nearest Rank Method)
const p95Index = Math.ceil(0.95 * sorted.length) - 1;
const p95 = sorted[Math.min(p95Index, sorted.length - 1)];

// 3. Filter > 800ms, extract unique, sort descending
const filteredUnique = Array.from(new Set(logs.filter(time => time <= 800)))
    .sort((a, b) => b - a);

return { p95, filteredUnique };
}

// Verification
console.log(analyzeResponseTimes([120, 450, 800, 230, 990, 150, 450, 300]));
// Output: { p95: 990, filteredUnique: [ 800, 450, 300, 230, 150, 120 ] }
```

Enterprise Senior SDET / Staff QA Architect Scoring Rubric

| Criterion | 1 - Unacceptable (Junior) | 3 - Satisfactory (Mid-Level) | 5 - Outstanding (Staff/Principal SDET) | |-----|-----|-----|-----| |

Technical Depth (Java/TS) | Struggles with memory model, hash collisions, or async execution mechanics. | Explains core syntax and basic OOP, but misses memory edge cases. | Deep mastery of JVM/V8 internals, thread safety, memory leak diagnosis, and protocol mechanics. | | **Framework Design** | Relies on static waits, monolithic scripts, hardcoded locators/data. | Writes basic POM, uses standard assertions, understands basic config. | Architects scalable multi-tenant frameworks, API data factories, custom fixtures, and isolated contexts. | | **Problem Solving & Coding** | Cannot solve basic array/map string manipulation without help. | Solves problem with brute-force $O(N^2)$ solution; basic null handling. | Writes clean $O(N \log N)$ or optimal code, handles edge cases, null guards, and cleanly structure methods. | | **CI/CD & Observability** | Treats CI as black box; unable to write YAML pipeline or debug build. | Can configure basic Jenkins job or GHA runner. | Designs matrix-sharded pipelines, artifact merging, quality gates, and Grafana flake dashboards. | | **Leadership & Strategic Impact** | Focuses strictly on manual/basic execution; blames environment for flakiness. | Triage test failures assigned to them; follows team processes. | Drives team-wide flake reduction (< 1%), establishes PR quality gates, coaches developers, and aligns metrics to business value. |

Passing Benchmark for Senior SDET at Deloitte:

- Minimum Score: **21 / 25 Total** (No single category below 3, with at least two categories scored 5).